

Object-Oriented Programming

const & mutable

How to freeze an object — and make exceptions

const

mutable

objects

class

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

Const Objects & Functions -- *Freezing an object — nothing can change*

const means: *"This object is FROZEN — all its data is protected from modification."*

Rules of const Objects

- const objects can only call const member functions
- Non-const functions CANNOT be called on const objects
- All member variables are protected from any changes
- Violation causes a compile-time error — caught immediately

C++

```
class Student {  
public:  
    string name;  
    int    age;  
  
    // const function ☒  
    void display() const {  
        cout << name;  
    }  
    // non-const function  
    void setAge(int a) {  
        age = a;  
    }  
};  
  
const Student s("Ali", 20);  
s.display(); // ☒ OK  
s.setAge(25); // ☐ ERROR
```

Sometimes you need to change just one thing...

The Problem —const Blocks You

Scenario: A **Library Book** is marked **const** because title/author never change. But you want to track how many times it was **accessed** — that count **MUST** change!

C++ — broken code

```
class Book {
public:
    string title;        // never changes – fine to const-protect
    string author;       // never changes – fine to const-protect
    int    accessCount;  // PROBLEM: needs to change every read!

    void read() const {
        accessCount++; // ✗ ERROR – cannot modify member in const function!
        cout << "Reading: " << title;
    }
};

int main() {
    const Book b("C++ Programming", "Stroustrup");
    b.read(); // ✗ COMPILE ERROR – even though title/author don't change
}
```

Compiler Error: assignment of member 'accessCount' in read-only object

mutable — Concept & Syntax *Granting special permission to one specific member*

mutable is a keyword placed before a member variable declaration. It says:

*"This ONE variable may be modified even inside a **const** function or on a **const** object."*

Syntax

```
mutable int accessCount;    // inside  
class  
mutable bool cacheValid;  
mutable double cachedResult;
```

Bypasses const



Only this variable escapes
the const protection

Others still frozen



All other members remain
fully const-protected

Compile-time rule



Enforced by the compiler
— zero runtime cost

Use responsibly



Only for bookkeeping,
never for core state

Applying mutable — Fixing the Book Class

✗ Before — Broken

```
class Book {  
    string title;  
    string author;  
    int    accessCount;  
    void read() const {  
        accessCount++; // ✗ ERROR  
    }  
};
```



✓ After — Fixed with mutable

```
class Book {  
    string title;  
    string author;  
    mutable int accessCount;  
    void read() const {  
        accessCount++; // ✓ OK  
    }  
};
```

main.cpp — works perfectly now

```
int main() {  
    const Book b("C++ Programming", "Stroustrup");  
    b.read();    // Output: Reading: C++ Programming | Access count: 1  
    b.read();    // Output: Reading: C++ Programming | Access count: 2  
    b.read();    // Output: Reading: C++ Programming | Access count: 3  
}
```

Real-Life Analogy -- *Museum painting — frozen, yet the counter ticks*



The Painting = `const`



- `title` = "Starry Night" — locked
- `artist` = "Van Gogh" — locked
- `medium` = "Oil on canvas" — locked



Visitor Counter = `mutable`

001,247

VISITORS

- `mutable` `visitorCount` changes every view ✓
- `mutable` `lastViewed` timestamp updated ✓

Key Insight: The painting is locked behind glass (`const`), but the visitor counter keeps ticking (`mutable`).

Practical Example: Caching -- *Store expensive results inside const objects*

Circle.cpp

```
class Circle {  
private:  
    double radius;  
    mutable double cachedArea;    // mutable cache  
    mutable bool areaCalculated;  // mutable flag  
public:  
    Circle(double r)  
        : radius(r), cachedArea(0), areaCalculated(false) {}  
  
    double getArea() const {  
        if (!areaCalculated) {  
            cout << "Calculating..." << endl;  
            cachedArea = 3.14159 * radius * radius;  
            areaCalculated = true;  
        } else {  
            cout << "Returning cache." << endl;  
        }  
        return cachedArea;  
    }  
};
```

Step-by-Step

- 1 **const Circle c(5.0);**
radius frozen. cachedArea=0, flag=false
- 2 **c.getArea() (1st call)**
Not cached → calculates
= $3.14 \times 25 = 78.54$
- 3 **cachedArea = 78.54**
Stored via mutable ✓
flag set to true
- 4 **c.getArea() (2nd call)**
Already cached →
returns instantly ✓
- 5 **c.getArea() (3rd call)**
Still cached →
no recalculation ✓

Calculating...
Returning cache.
Returning cache.

What mutable Does NOT Mean — Misconceptions

X

MYTH: The whole object is no longer const

FACT: Only the mutable member loses protection; all others stay frozen

X

MYTH: You can change any member you want

FACT: Only members explicitly declared mutable can be changed in const functions

X

MYTH: mutable = bad design / anti-pattern

FACT: Perfectly valid for bookkeeping (counters, caches, mutexes, logs)

Side-by-Side Comparison — normal vs mutable Members

C++

```
class Example {  
public:  
    int    normalData;        // protected by const – CANNOT change in const function  
    mutable int mutableData;    // EXCEPTION – CAN change even in const function  
  
    void constFunc() const {  
        normalData = 5;    // ✗ ERROR – const function cannot modify normal member  
        mutableData = 5;    // ✓ OK – mutable bypasses const restriction  
    }  
};  
  
int main() {  
    const Example e;  
    e.constFunc();    // ✓ works – only mutableData changes inside  
}
```

Feature	Normal Member	mutable Member
In const function	✗ CANNOT modify	✓ CAN modify
On const object	✗ CANNOT modify	✓ CAN modify
Requires keyword	(none — default)	mutable keyword
Protects logic	Yes — core state	No — helper only

When to Use mutable — Valid Use Cases



Access / Hit Counters

Track how many times a method is called on a const object

Example: `accessCount`, `hitCount`, `readCount`



Caching Results

Store expensive computations so they don't repeat

Example: `cachedArea`, `cachedHash`, `cachedDistance`



Logging / Debugging

Record events or errors inside const functions

Example: `lastError`, `logBuffer`, `debugInfo`



Mutex Locks (Threading)

Lock/unlock a mutex inside const methods for thread safety

Example: `mutable std::mutex mtx;`



Timestamps

Update `lastAccessed` or `lastModified` time on a const object

Example: `mutable time_t lastAccessed;`



State Flags

Boolean flags that track internal state, not external behavior

Example: `mutable bool initialized`, `mutable bool dirty`;

✗ Never use mutable for: core data (like name, price, balance) — only use it for internal bookkeeping that does not affect the object's logical state.

Summary — const & mutable in C++

const Object

Freezes ALL member variables
Only const functions can be called
Violations caught at compile time

const Function

Declared with const after ()
Cannot modify any member
Can be called on const objects

mutable Keyword

Placed before member declaration
Allows change even in const context
Only that ONE variable is exempt

Syntax

```
mutable int counter;  
mutable bool cacheValid;  
mutable double result;
```

Valid Uses

Counters, caches, logs, mutexes
Timestamps, state flags
Anything that is bookkeeping only

Golden Rule

mutable should NEVER change
the logical/observable state
of the object — only internals!

Dynamic Memory Allocation in C++

new · delete · Heap · Stack · Memory Leaks · Smart Pointers

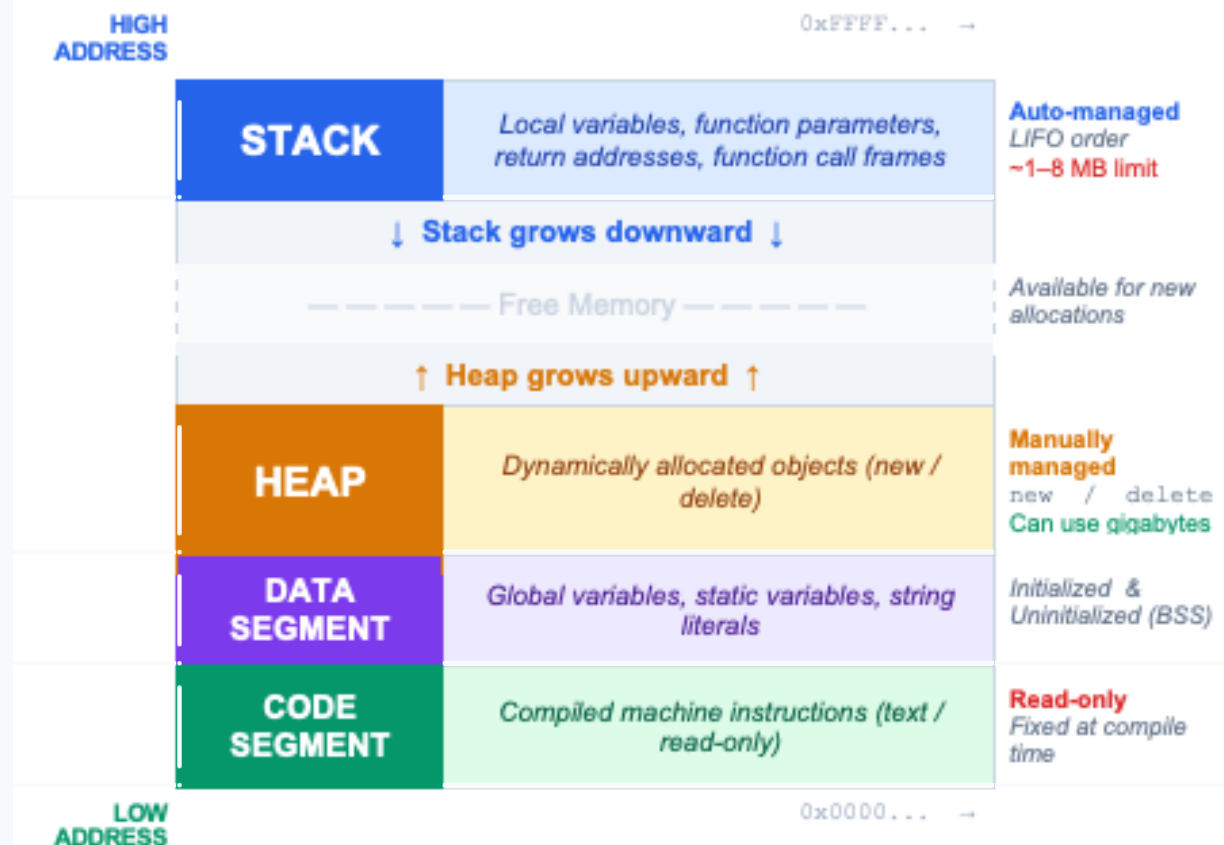


RAM Memory

Stack | Heap | Code

Memory Layout

Stack vs Heap
where do variables live?

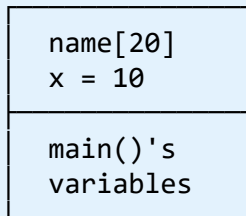


Stack — Automatic Memory

```
void greet() {  
    int x = 10;           // pushed onto stack when greet() is called  
    char name[20] = "Ali"; // also on stack  
    // ...  
}                        // x and name automatically destroyed here  
                        // popped off the stack — no action needed
```

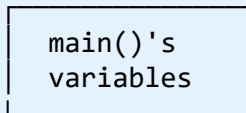
How It Works
...

main() calls greet()



← top of stack
greet()'s variables

greet() returns



← greet's frame GONE — popped automatically

The stack works like a **stack of plates** — you add to the top, remove from the top. LIFO order.

? Why Do We Need Dynamic Memory?

The Stack has limits. When you don't know the size of data at compile time — or when data must outlive a function — you need the **Heap**.



Unknown Size

```
int arr[?]; // ✗ can't do this!  
// Must decide size at runtime!
```



Large Data

Stack ~1-8 MB only.
Heap = gigabytes available.



Data Structures

Linked lists, trees, graphs
all need dynamic allocation.



Outlive Functions

Heap memory persists beyond
the function that created it.

Heap — Dynamic Memory

The heap is a large **free pool** of memory.

Chunk of memory is requested manually, which stays until explicitly released.

```
void example() {  
    int* p = new int(42);    // allocated on HEAP  
    // p itself (the pointer) is on the stack  
    // the int value 42 lives on the heap  
  
    // ... use p ...  
  
    delete p;               // YOU must free it – won't happen automatically  
    p = nullptr;  
}  
...
```


Stack vs Heap Memory

STACK (Automatic)

← High Address

```
int x = 5;
```

```
double d;
```

```
someFunction()
```

```
main()
```

← Low Address

↓ *grows downward* | *Auto managed* | *~1-8 MB limit*

Stack: auto, fast, limited size

VS

HEAP (Free Store)

← Low Address

```
new int(5)
```

```
new double
```

```
new MyClass()
```

```
new int[100]
```

← High Address

↑ *grows upward* | *Manual managed* | *Gigabytes!*

Heap: manual, flexible, large — but YOU must manage it!

Heap — Dynamic Memory

```
// — STACK —————  
int* stackProblem() {  
    int x = 42;    // x lives on stack inside this function  
    return &x;    // ✗ DANGER — returning address of stack variable  
}                // x is DESTROYED here when function returns  
                // caller gets a dangling pointer!  
  
// — HEAP —————  
int* heapSolution() {  
    int* p = new int(42); // 42 lives on HEAP — persists after return  
    return p;            // ✓ safe — heap memory survives  
}                        // caller must delete p when done!  
  
int main() {  
    int* bad = stackProblem(); // undefined behaviour — x is gone  
    int* good = heapSolution(); // works — 42 still exists on heap  
    cout << *good;            // 42  
    delete good;              // caller's responsibility  
}
```

Size Difference

```
// ----- STACK – small limit -----  
void stackExample() {  
    int arr[1000000];           // ✗ likely STACK OVERFLOW (~4 MB on stack)  
}
```

Stack overflow happens when:

- Too many recursive calls (each call adds a frame)
- Very large local arrays declared inside functions

```
void infinite() {  
    int data[1000];  
    infinite();                 // each call adds 1000 ints to stack  
}                               // → stack overflow crash!
```

```
// ----- HEAP – can be large -----  
void heapExample() {  
    int* arr = new int[1000000]; // ✓ fine – heap has gigabytes  
    // ... use arr ...  
    delete[] arr;  
}
```

The memory manager must:

1. Search the free list for a block big enough
2. Mark it as used
3. Return the address
4. On delete — mark it free, possibly merge adjacent blocks

Memory Distribution

```
#include <iostream>
using namespace std;

int globalVar = 100;

int main() {
    int localVar = 10;
    int* ptr = new int(50);

    return 0;
}
```

- globalVar → Data segment
- localVar → Stack
- ptr → Stack
- *ptr (50) → Heap

Stack vs Heap

	STACK	HEAP
Managed by	Compiler/OS	Programmer
Allocation	Automatic	Manual (<code>new</code>)
Deallocation	Automatic (scope end)	Manual (<code>delete</code>)
Size	~1 MB - 8 MB	~GB (limited by RAM)
Speed	Very fast	Slower
Fragmentation	None	Can fragment
Overflow possible?	YES (stack overflow)	YES (out of memory)
Lifetime	Scope-based	Until <code>delete</code> called
Order	LIFO	Random / flexible
Thread safety	Each thread has own	Shared – needs care

When to Use Which

Use STACK when:

- ✓ Size is known at compile time
- ✓ Variable only needed inside current function/scope
- ✓ Small to medium **size** (not megabytes)
- ✓ You want automatic cleanup – no **delete** needed
- ✓ Simple types: **int**, **double**, small arrays, small objects

Use HEAP when:

- ✓ Size is known only at **runtime** (user input, file size)
- ✓ Object must outlive the function that created it
- ✓ Very large **data** (arrays of millions of elements)
- ✓ You need to resize memory during program execution
- ✓ Linked lists, trees, graphs – pointer-based structures



The new Operator – Syntax & What It Does

Allocating memory on the heap at runtime

- **new** allocates memory on the **heap** at runtime, calls the constructor automatically, and returns a **typed pointer**.

new operator:

- The **new** operator automatically
 - creates an object/array/variable of proper size
 - for object it calls the constructor
 - returns a pointer of the correct type
- If no space is available in memory, then it will return an error

In run time when an object is created, it has no name and its address is used in programs

```
int *gradesArray = new int[ 10 ];  
double *ptr = new double( 3.14159 );  
Time *timePtr = new Time;  
Time *timePtr = new Time( 12, 0, 0 );
```



The new Operator — Syntax & What It Does

Allocating memory on the heap at runtime

new allocates memory on the **heap** at runtime, calls the constructor automatically, and returns a **typed pointer**.

C++ Syntax

```
// — Single variable —————
int* p = new int;           // uninitialized
int* p = new int(42);       // init = 42
double* d = new double(3.14);

// — Array (size can be a variable!) ———
int n = 10;
int* arr = new int[n];     // runtime size!

// — Class Object —————
Time* t = new Time;         // default ctor
Time* t = new Time(12,0,0); // param ctor

// — nothrow (no exception on fail) ———
int* p = new(nothrow) int[99999];
if (p == nullptr) cout << "Failed!";
```



Correct Size

sizeof(type) bytes allocated



Calls Constructor

Object fully initialized



Returns T*

Correct typed pointer returned



On Failure

Throws std::bad_alloc



Include <new> for nothrow

The delete operator

Releasing heap memory — your responsibility!

Delete operator

- To destroy an object dynamically and free its space, we must use delete operator

```
delete timePtr;  
delete [] gradesArray;
```

- It dynamically deallocates the memory
- **delete** operator calls destructor of an object to be deleted
- It deallocates the memory associated with the object
- This memory can be reused to allocate other objects
- If there is a pointer to array of objects
 - It calls destructor for each object in array
 - Then deallocates memory



The delete Operator — Syntax & Rules

delete calls the destructor, then frees the heap memory. You **MUST** pair every **new** with **delete** — or memory leaks!

C++ — delete

```
// — Single object —————
int* p = new int(99);
cout << *p;           // Output: 99
delete p;             // ✓ free memory
p = nullptr;          // ✓ avoid dangling ptr

// — Arrays: MUST use delete[] —————
int* arr = new int[5];
delete arr;            // ✗ WRONG — undefined!
delete[] arr;          // ✓ correct for arrays
arr = nullptr;

// — Class objects —————
Time* t = new Time(12,0,0);
delete t;              // ~Time() called first, then free
```

Golden Rules

- ✓ Always delete what you new
- ✓ Always delete[] what you new[]
- ✓ Set pointer = nullptr after delete

✗ Never delete a stack variable

✗ Never mix delete with new[]

✗ Never delete the same ptr twice

delete does: 1) calls dtor 2) frees memory 3) does NOT zero the pointer!



Dynamic Arrays — `new[]` and `delete[]`

Arrays whose size is decided at runtime!

Problem: `int arr[500]` wastes space or runs out. Dynamic arrays let the **user decide the size** at runtime!

grades.cpp

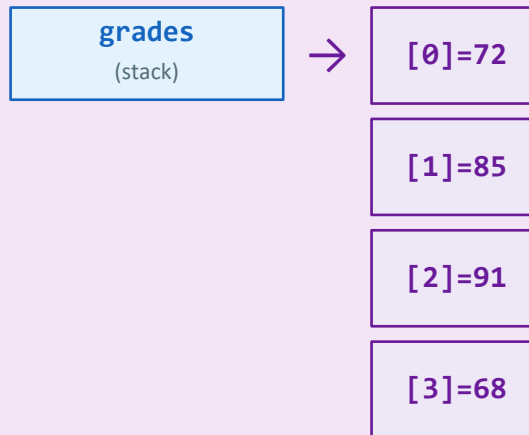
```
#include <iostream>
using namespace std;
int main() {
    int n;
    cout << "How many grades? ";
    cin >> n;          // user decides size

    int* grades = new int[n]; // ← runtime!

    for (int i=0; i<n; i++) {
        cout << "Grade " << i+1 << ": ";
        cin >> grades[i];
    }

    delete[] grades; // ✓ use delete[]!
    grades = nullptr;
    return 0; }
```

Memory (n=4)



Heap →



Key Rule: `new int[n]` $\leftrightarrow$ `delete[] ptr` (NEVER plain delete)



Dynamic Objects — Constructor & Destructor

Student.cpp

```
class Student {  
    string name; int age;  
public:  
    Student(string n, int a) {  
        name=n; age=a;  
        cout << "✅ Created: " << name;  
    }  
    ~Student() {  
        cout << "🗑 Destroyed: " << name;  
    }  
    void print() const {  
        cout << name << ", age " << age;  
    }  
};  
  
int main() {  
    Student* s = new Student("Ali", 20);  
    // Output: ✅ Created: Ali  
    s->print();    // use -> not . for pointers!  
    delete s;      // calls ~Student()  
    // Output: 🗑 Destroyed: Ali  
    s = nullptr;  
}
```

new Student(...)

1

Memory allocated on heap
Constructor called auto

s->print()

2

Use -> operator (not .)
to access via pointer

delete s

3

Destructor called first
Then memory freed

s = nullptr

4

Safety: prevents crashes
from dangling pointer

Dynamic Memory — new and delete with Classes

C++

```
#include <new>

Time *timePtr = new Time;           // allocate - calls default constructor
Time *timePtr = new Time(12, 0, 0); // allocate - calls parameterized constructor
int *arr      = new int[10];        // allocate array of 10 ints

delete timePtr;                     // calls destructor + frees memory
delete[] arr;                       // calls destructor for each element

// Safe allocation with nothrow (no exception on failure)
int *p = new(nothrow) int[999999];
if(p == nullptr) cout << "Allocation failed!";
```

new Operator

- Creates object of proper size
- Calls constructor automatically
- Returns pointer of correct type
- Throws `bad_alloc` if out of memory

delete Operator

- Calls destructor of object
- Frees allocated memory
- Memory can be reused
- `delete[]` for arrays

Best Practices

- Always delete what you new
- Always `delete[]` what you `new[]`
- Set pointer to `nullptr` after delete
- Use `nothrow` for large allocations

When to `#include <new>`

`new` and `delete` are **built-in operators** -

Keywords of the C++ language itself — They don't need any header to work.

In C++, when something goes wrong at runtime, a function can **throw an exception** — it's like the program raising an alarm "*something failed, I can't continue normally.*"

- e.g. if `new` cannot find enough memory, it **throws** a `bad_alloc` exception. The program immediately jumps to the nearest catch block. If there is no catch, the **program crashes**.

`#include <new>` gives you access to the **advanced versions** of `new` and `delete`:

- `nothrow` tells `new`: "*do NOT throw an exception if you fail — just return `nullptr` instead.*"

```
// — DEFAULT (throws) _____  
int* p1 = new int[999999999];  
// If fails: throws bad_alloc → crash if not caught
```

```
// — NOTHROW _____  
int* p2 = new(nothrow) int[999999999];  
// If fails: returns nullptr → you check it manually  
if (p2 == nullptr) { /* handle */ }  
...  
...
```

	<code>new</code> (default)	<code>new(nothrow)</code>
On success	Returns valid pointer	Returns valid pointer
On failure	Throws <code>bad_alloc</code>	Returns <code>nullptr</code>
Need try/catch?	Yes (recommended)	No — just check <code>nullptr</code>
Header needed	No (for basic use)	Yes — <code>#include <new></code>
Speed	Slightly slower	Slightly faster
Use when	Critical allocations	Performance code

```
..
```



Common Memory Problems — Causes & Fixes

The bugs that haunt every C++ programmer...



Memory Leak

Allocated memory never freed. Program eats RAM until crash.

✗ BAD

```
int* p = new int(5);  
// ... p goes out of scope  
// memory never freed! 💧
```

✓ FIX

```
int* p = new int(5);  
// ... use p ...  
delete p; p=nullptr; // ✓
```



Dangling Pointer

Accessing memory AFTER it was deleted — crashes or garbage data.

✗ BAD

```
int* p = new int(10);  
delete p;  
cout << *p; // ✗ CRASH!
```

✓ FIX

```
int* p = new int(10);  
delete p;  
p = nullptr; // safe ✓
```



Double Delete

Calling delete twice on the same pointer — corrupts the heap.

✗ BAD

```
int* p = new int(7);  
delete p; // first  
delete p; // ✗ UNDEFINED!
```

✓ FIX

```
int* p = new int(7);  
delete p;  
p=nullptr; // delete(null)=safe ✓
```



"Every new has a delete. Every new[] has a delete[]. Always set deleted ptrs to nullptr."

Smart Pointers — Modern C++ Memory Safety

Smart pointers (`<memory>`) automatically call **delete** when they go out of scope — **zero memory leaks!**

`unique_ptr`

Single owner only
Auto-deletes on scope exit
Cannot be copied

```
#include <memory>

unique_ptr<int> p =
    make_unique<int>(42);

// auto-deleted ✓
```

`shared_ptr`

Multiple owners allowed
Reference counted
Deletes when count = 0

```
shared_ptr<int> p1 =
    make_shared<int>(10);

shared_ptr<int> p2=p1;

// deleted when both gone
```

`weak_ptr`

Non-owning observer
Breaks circular references
Must `lock()` to access

```
shared_ptr<int> sp =
    make_shared<int>(5);

weak_ptr<int> wp = sp;

// wp doesn't own it
```

✓ **Best Practice:** Prefer `make_unique` / `make_shared` over raw `new` in modern C++11+ code!



Summary — Dynamic Memory in C++

new Operator

Allocates heap, calls ctor,
returns typed pointer. Include `<new>`.

delete Operator

Calls dtor, frees memory.
Pair every new $\rightarrow$ delete.

Dynamic Arrays

`new int[n]` $\leftrightarrow$ `delete[] ptr`
Runtime-sized arrays!

Memory Problems

Leaks, dangling ptrs, double-delete
— all preventable!

Classes

`new` calls ctor, `delete` calls dtor.
Use `->` with object pointers.

Smart Pointers

`unique_ptr`, `shared_ptr` `<memory>`
Prefer in C++11+ code.